CPPDescent

91cb4bc (with uncommitted changes)

Generated by Doxygen 1.9.6

Chapter 1

CPP-Descent

Part of the "Software Development for Computing Systems" course of the Department of Informatics and Telecommunications, UoA for the first semester of the academic year 2023-2024.

A library that implements the KNN-Graph creation and search algorithms described in [1], [2].

1.1 Dependencies

For local development, you will need `Make` and `CMake`. To install them on an apt-based linux distro, run the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install make
$ sudo apt install cmake
```

1.1.1 Minor dependencies

- The project uses Google Test for its testing needs. Gtest is used as a git submodule of this repo, that is downloaded by CMake the first time you clone. That means that nothing extra needs to be installed to run the tests.
- If you would like to generate the documentation, you will need `doxygen` installed.

1.2 Code Structure

In the wiki, you can find documentation generated by Doxygen. It is also available in pdf form.

1.2.1 Code Coverage

This project uses LCOV for its code coverage needs. The results are uploaded upon commit here.

1.3 Contributors

- Konstantinos Chousos (1115202000215)
- Anastasios-Fedon Seitanidis (1115202000179)

1.4 Bibliography

[1] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in *Proceedings of the 20th international conference on World wide web*, in WWW ’11. New York, NY, USA: Association for Computing Machinery, Mar. 2011, pp. 577–586. doi: 10.1145/1963405.1963487.

[2] “How PyNNDescent works — pynndescent 0.5.0 documentation.” Accessed: Oct. 10, 2023. [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 4

Class Documentation

4.1 cppdescent Class Reference

A sample function for the cppdescent class, used to test the correctness of the CMake files.

```
#include <cppdescent.hpp>
```

Public Member Functions

- int hello (void)

    Prints "Hello World".

4.1.1 Detailed Description

A sample function for the cppdescent class, used to test the correctness of the CMake files.

4.1.2 Member Function Documentation

4.1.2.1 hello()

```
int cppdescent::hello (
            void )
```

Prints "Hello World".

Returns

int exit code

The documentation for this class was generated from the following files:

- cppdescent.hpp
- cppdescent.cpp

4.2 Graph Class Reference

Public Member Functions

- **Graph** (CompareFunc compare, DestroyFunc destroy)
- int **getSize** ()
- void **insertVertex** (Pointer vertex)
- List * getVertices ()

    The returned list needs to be deleted, but the list's elements are pointers the the vector's elements, so the list shouldn't have a `destroyValue`*.*
- void **removeVertex** (Pointer vertex)
- void **insertEdge** (Pointer vertex1, Pointer vertex2, int weight)
- void **removeEdge** (Pointer vertex1, Pointer vertex2)
- int **getWeight** (Pointer vertex1, Pointer vertex2)
- List * **getAdjacent** (Pointer vertex)
- void **setHashFunction** (HashFunc hash)
- CompareFunc **getCompare** ()
- DestroyFunc **getDestroy** ()
- HashFunc **getHash** ()

4.2.1 Member Function Documentation

4.2.1.1 getVertices()

```
List * Graph::getVertices ( )
```

The returned list needs to be deleted, but the list's elements are pointers the the vector's elements, so the list shouldn't have a `destroyValue`.

Returns

List*

The documentation for this class was generated from the following files:

- ADTGraph.hpp
- ADTGraph.cpp

4.3 GraphVertexPair Class Reference

Public Member Functions

- **GraphVertexPair** (Graph *owner, Pointer vertex1, Pointer vertex2)
- Pointer **getVertex1** ()
- Pointer **getVertex2** ()
- Graph * **getOwner** ()

The documentation for this class was generated from the following file:

- ADTGraph.hpp

4.4 List Class Reference

ADT List.

`#include <ADTList.hpp>`

Public Member Functions

- List (DestroyFunc value=nullptr)

    Construct a new List object.
- ∼List ()

    Destroy the List object.
- void insertNext (ListNode ∗node, Pointer value)

    Adds a node after the given node.
- void removeNext (ListNode ∗node)

    Removes the node after the given node.
- Pointer find (Pointer value, CompareFunc compare)

    Finds the first value equal to the value parameter.
- DestroyFunc setDestroyValue (DestroyFunc value)

    Set the Destroy Value object to a new value.
- int getSize ()

    Get the size of the List.
- int increaseSize ()

    Increase the list size.
- int decreaseSize ()

    Decrease the list size.
- ListNode ∗ getHead ()

    Getter for the head of the List.
- ListNode ∗ getTail ()

    Getter for the tail of the List.
- ListNode ∗ next (ListNode ∗node)

    Returns the node after the given one.
- Pointer nodeValue (ListNode ∗node)

    Returns the value of the node.
- ListNode ∗ findNode (Pointer value, CompareFunc compare)

    Finds the first node that has value equal to the value parameter.

4.4.1 Detailed Description

ADT List.

An Abstract Data Type List with linear access to data, and addition and removal to/from arbitrary positions.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 List()

```
List::List (
            DestroyFunc value = nullptr )
```

Construct a new List object.

Parameters

destroyValue	If destroyValue != nullptr, the destroyValue function will be called each time a node is removed.

head is a virtual node that is always present in the list.

Parameters

destroyValue	The destroy function for the values of the list. Defaults to nullptr.

4.4.2.2 ∼List()

```
List::∼List ( )
```

Destroy the List object.

If the destroyValue function is not nullptr, it is called for every value.

4.4.3 Member Function Documentation

4.4.3.1 decreaseSize()

```
int List::decreaseSize ( )
```

Decrease the list size.

Returns

int The updated size.

4.4.3.2 find()

```
Pointer List::find (
            Pointer value,
            CompareFunc compare )
```

Finds the first value equal to the value parameter.

Find the first value equal to the value parameter.

Parameters

value	The value based on which the search is done.
compare	A pointer to the function that will compare the node values.

Returns

Pointer A pointer to the resulting node.

Parameters

value	The value to search for.
compare	A pointer to the compare function.

Returns

Pointer A pointer to found value.

4.4.3.3 findNode()

```
ListNode * List::findNode (
            Pointer value,
            CompareFunc compare )
```

Finds the first node that has value equal to the value parameter.

Find the first node with value equal to the value parameter.

Parameters

value	The value to search.
compare	The compare function to use.

Returns

ListNode∗ A pointer to the resulting node.

Parameters

value	The value to search for.
compare	A pointer to the compare function.

Returns

ListNode∗ A pointer to the found node.

4.4.3.4 getHead()

`ListNode * List::getHead ( )`

Getter for the head of the List.

Get the first element of the List.

Returns

ListNode∗ The head.

Returns

ListNode∗ Pointer to the first element.

4.4.3.5 getSize()

`int List::getSize ( )`

Get the size of the List.

Get the size of the list.

The size is updated upon entry/removal of nodes, so the retrieval of the size is O(1).

Returns

int

Returns

int The size.

4.4.3.6 getTail()

`ListNode * List::getTail ( )`

Getter for the tail of the List.

Get the last element of the list.

Returns

ListNode∗ The tail.

If the list is empty, the virtual head is the tail, so we return nullptr.

Returns

ListNode∗ Pointer to the last element.

4.4.3.7 increaseSize()

```
int List::increaseSize ( )
```

Increase the list size.

Returns

int The updated size.

4.4.3.8 insertNext()

```
void List::insertNext (
            ListNode * node,
            Pointer value )
```

Adds a node *after* the given node.

Insert a new element after the node.

Parameters

node	The node that the new node will be placed after.
value	The value of the new node.

Parameters

node	The node after which the element is inserted.
value	The value of the new element.

4.4.3.9 next()

```
ListNode * List::next (
            ListNode * node )
```

Returns the node after the given one.

Get the next node.

Parameters

node	

Returns

ListNode* A pointer to the next node.

Parameters

node	

Returns

ListNode*

4.4.3.10 nodeValue()

```
Pointer List::nodeValue (
            ListNode * node )
```

Returns the value of the node.

Get the node's value.

Parameters

node	

Returns

Pointer A generic pointer to the value.

Parameters

node	

Returns

Pointer

4.4.3.11 removeNext()

```
void List::removeNext (
            ListNode * node )
```

Removes the node *after* the given node.

Removes the node after the given node.

Parameters

node	The node before the node to be removed.

Parameters

node	The node after which the element is removed.

4.4.3.12 setDestroyValue()

```
DestroyFunc List::setDestroyValue (
            DestroyFunc value )
```

Set the Destroy Value object to a new value.

Set the destroy value function.

Parameters

value	A pointer to the new destroy funciton.

Returns

DestroyFunc A pointer to the old destroy function.

Parameters

value	The new destroy function.

Returns

DestroyFunc The old destroy function.

The documentation for this class was generated from the following files:

- ADTList.hpp
- ADTList.cpp

4.5 ListNode Class Reference

Node of a list.

```
#include <ADTList.hpp>
```

Public Member Functions

- ListNode ()
  Construct a new List Node object that is empty.
- ListNode (Pointer value)
  Construct a new List Node object with value 'value'.
- void setNext (ListNode *next)
  Setter for the next node.
- ListNode * getNext ()
  Getter for the next node.
- Pointer getValue ()
  Getter for the value of the node.

4.5.1 Detailed Description

Node of a list.

A list node is represented by its value. It also holds a pointer to the next node in the list.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 ListNode() [1/2]

```
ListNode::ListNode ( )  [inline]
```

Construct a new List Node object that is empty.

4.5.2.2 ListNode() [2/2]

```
ListNode::ListNode (
            Pointer value )  [inline]
```

Construct a new List Node object with value 'value'.

Parameters

value	a generic pointer to the value of the node.

4.5.3 Member Function Documentation

4.5.3.1 getNext()

```
ListNode * ListNode::getNext ( )
```

Getter for the next node.

Returns

ListNode∗ The pointer to the next node.

4.5.3.2 getValue()

```
Pointer ListNode::getValue ( )
```

Getter for the value of the node.

Returns

Pointer A generic pointer to the value.

4.5.3.3 setNext()

```
void ListNode::setNext (
            ListNode * next )
```

Setter for the next node.

Parameters

next	The pointer to the next node.

The documentation for this class was generated from the following files:

- ADTList.hpp
- listNode.cpp

4.6 Map Class Reference

Public Member Functions

- Map (CompareFunc compare, DestroyFunc destroyKey, DestroyFunc destroyValue)

    Construct a new Map:: Map object.
- ∼Map ()

Destroy the Map:: Map object.
- void insert (Pointer key, Pointer value)

  Add a new pair (key --> value) in the map.
- bool remove (Pointer key)

  Remove from the map the pair (key --> value) that corresponds to the given key.
- Pointer **find** (Pointer key)
- void rehash ()

  Extend the hash table in case the load factor exceeds the predefined threshold.
- void **setCapacity** (int capacity)
- void **setDestroyKey** (DestroyFunc destroyKey)
- void **setDestroyValue** (DestroyFunc destroyValue)
- void **setHashFunction** (HashFunc hash)
- MapNode ** **getArray** ()
- int **getCapacity** ()
- int **getSize** ()
- int **getDeleted** ()
- DestroyFunc **getDestroyKey** ()
- DestroyFunc **getDestroyValue** ()
- HashFunc **getHashFunction** ()
- MapNode * getFirst ()
- MapNode * getNext (MapNode *node)
- MapNode * findNode (Pointer key)

4.6.1 Constructor & Destructor Documentation

4.6.1.1 Map()

```
Map::Map (
            CompareFunc compare,
            DestroyFunc destroyKey,
            DestroyFunc destroyValue )
```

Construct a new Map:: Map object.

Parameters

compare	A compare function to keep the map sorted
destroyKey	A destroy function for the key of every node
destroyValue	A destroy function fro the balue of every node

4.6.1.2 ∼Map()

```
Map::∼Map ( )
```

Destroy the Map:: Map object.

4.6.2 Member Function Documentation

4.6.2.1 findNode()

```
MapNode * Map::findNode (
            Pointer key )
```

Parameters

key	

Returns

MapNode*

4.6.2.2 getFirst()

```
MapNode * Map::getFirst ( )
```

Returns

MapNode*

4.6.2.3 getNext()

```
MapNode * Map::getNext (
            MapNode * node )
```

Parameters

node	

Returns

MapNode*

4.6.2.4 insert()

```
void Map::insert (
            Pointer key,
            Pointer value )
```

Add a new pair (key --> value) in the map.

Parameters

key	
value	

4.6.2.5 rehash()

```
void Map::rehash ( )
```

Extend the hash table in case the load factor exceeds the predefined threshold.

Parameters

map	The map whose hash table needs to be rehashed

4.6.2.6 remove()

```
bool Map::remove (
            Pointer key )
```

Remove from the map the pair (key --> value) that corresponds to the given key.

Returns

true he node with the given key was removed successfully

false he node with the given key was not removed

The documentation for this class was generated from the following files:

- ADTMap.hpp
- ADTMap.cpp

4.7 MapNode Class Reference

Public Member Functions

- **MapNode** (Pointer key, Pointer value, State state)
- void **setKey** (Pointer key)
- void **setValue** (Pointer value)
- void **setState** (State state)
- Pointer **getKey** ()
- Pointer **getValue** ()
- State **getState** ()

The documentation for this class was generated from the following file:

- ADTMap.hpp

4.8 PQueue Class Reference

ADT Priority Queue.

```
#include <ADTPQueue.hpp>
```

Public Member Functions

- PQueue (CompareFunc compare, DestroyFunc destroyValue, Vector ∗values)
  Construct a new PQueue object.
- ∼PQueue ()
  Destroy the PQueue object.
- int getSize ()
  Get the size of the priority queue.
- Pointer getMax ()
  Get the max element of the queue.
- void insert (Pointer value)
  Insert a new element to the queue.
- void removeMax ()
  Remove the max of the queue.
- DestroyFunc setDestroyValue (DestroyFunc destroyValue)
  Set the Destroy Value object.
- Pointer **nodeValue** (int nodeId)
- void **nodeSwap** (int nodeId1, int nodeId2)
- void **bubbleUp** (int nodeId)
- void **bubbleDown** (int nodeId)
- void **naiveHeapify** (Vector ∗values)

4.8.1 Detailed Description

ADT Priority Queue.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 PQueue()

```
PQueue::PQueue (
            CompareFunc compare,
            DestroyFunc destroyValue,
            Vector * values )
```

Construct a new PQueue object.

Uses the `compare` function to compare its elements.

If `destroyValue` != nullptr, the `destroyValue` function is called upon removal of an element.

If `values` != nullptr, then the priority queue will be initialized with the values of the vector `values`.

Parameters

compare	
destroyValue	
values	

4.8.2.2 ∼PQueue()

```
PQueue::~PQueue ( )
```

Destroy the PQueue object.

4.8.3 Member Function Documentation

4.8.3.1 getMax()

```
Pointer PQueue::getMax ( )
```

Get the max element of the queue.

Returns

Pointer A generic pointer to the max element.

4.8.3.2 getSize()

```
int PQueue::getSize ( )
```

Get the size of the priority queue.

Returns

int

4.8.3.3 insert()

```
void PQueue::insert (
            Pointer value )
```

Insert a new element to the queue.

Parameters

value	The value of the new element to be inserted.

4.8.3.4 removeMax()

```
void PQueue::removeMax ( )
```

Remove the max of the queue.

4.8.3.5 setDestroyValue()

```
DestroyFunc PQueue::setDestroyValue (
            DestroyFunc destroyValue )
```

Set the Destroy Value object.

Parameters

destroyValue	

Returns

DestroyFunc

The documentation for this class was generated from the following files:

- ADTPQueue.hpp
- ADTPQueue.cpp

4.9 Vector Class Reference

ADT Vector.

```
#include <ADTVector.hpp>
```

Public Member Functions

- Vector (int size, DestroyFunc destroyValue)
    Construct a new Vector object.
- ∼Vector ()
    Destroy the Vector object.
- int getSize ()
    Get the current size of the Vector.
- void insertLast (Pointer value)
    Inserts a new element at the end of the vector.
- int removeLast ()
    Removes the last element of the vector.
- Pointer getAt (int pos)
    Get the value at the specified position of the vector.
- int setAt (int pos, Pointer value)
    Set the value at the specified position of the vector.
- Pointer find (Pointer value, CompareFunc compare)
    Find the first element with value equal to value.
- DestroyFunc setDestroyValue (DestroyFunc destroyValue)
    Set the Destroy Value.
- vectorNode ∗ first ()
    Get the first node of the vector.
- vectorNode ∗ last ()
    Get the last node of the vector.
- vectorNode ∗ next (vectorNode ∗node)
    Get the next node of the vector, after the one given.
- vectorNode ∗ previous (vectorNode ∗node)
    Get the previous node of the vector, before the one given.
- Pointer nodeValue (vectorNode ∗node)
    Get the value of the node.
- vectorNode ∗ findNode (Pointer value, CompareFunc compare)
    Find the first node with value equal to value.

4.9.1 Detailed Description

ADT Vector.

An ADT Vector using a dynamic array for smart resizing.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 Vector()

```
Vector::Vector (
            int size,
            DestroyFunc destroyValue )
```

Construct a new Vector object.

The newly created Vector will be of size size and its elements will be initialized as nullptr.

If destroyValue is not nullptr, it will be called on each element when it is removed from the Vector.

Parameters

size	The initial size of the Vector.
destroyValue	The function to be called on each element when it is removed from the Vector.

4.9.2.2 ∼Vector()

```
Vector::∼Vector ( )
```

Destroy the Vector object.

Deletes all allocated memory of the Vector.

4.9.3 Member Function Documentation

4.9.3.1 find()

```
Pointer Vector::find (
            Pointer value,
            CompareFunc compare )
```

Find the first element with value equal to value.

Parameters

value	The value to look for.
compare	The function to be used for comparison.

Returns

Pointer A pointer to the found value.

4.9.3.2 findNode()

```
vectorNode * Vector::findNode (
             Pointer value,
             CompareFunc compare )
```

Find the first node with value equal to value.

Parameters

value	The value to look for.
compare	The function to be used for comparison.

Returns

vectorNode The resulting node.

4.9.3.3 first()

```
vectorNode * Vector::first ( )
```

Get the first node of the vector.

Returns

vectorNode

4.9.3.4 getAt()

```
Pointer Vector::getAt (
             int pos )
```

Get the value at the specified position of the vector.

Parameters

pos	The position to look for.

Returns

Pointer A pointer to the value at the specified position.

4.9.3.5 getSize()

```
int Vector::getSize ( )
```

Get the current size of the Vector.

Returns

int The size of the Vector.

4.9.3.6 insertLast()

```
void Vector::insertLast (
            Pointer value )
```

Inserts a new element at the end of the vector.

Parameters

value	The value of the new element.

If we are at the last element, we create a new array double the size.

4.9.3.7 last()

```
vectorNode * Vector::last ( )
```

Get the last node of the vector.

Returns

vectorNode

4.9.3.8 next()

```
vectorNode * Vector::next (
            vectorNode * node )
```

Get the next node of the vector, after the one given.

Parameters

node	The node to get the next of.

Returns

vectorNode

4.9.3.9 nodeValue()

```
Pointer Vector::nodeValue (
            vectorNode * node )
```

Get the value of the node.

Parameters

node	

Returns

Pointer

4.9.3.10 previous()

```
vectorNode * Vector::previous (
            vectorNode * node )
```

Get the previous node of the vector, before the one given.

Parameters

node	The node to get the previous of.

Returns

vectorNode

4.9.3.11 removeLast()

```
int Vector::removeLast ( )
```

Removes the last element of the vector.

Returns

int -1 if the function fails, else 0.

4.9.3.12 setAt()

```
int Vector::setAt (
            int pos,
            Pointer value )
```

Set the value at the specified position of the vector.

Parameters

pos	The position to edit.
value	The new value.

Returns

int -1 if the function fails, else 0.

4.9.3.13 setDestroyValue()

```
DestroyFunc Vector::setDestroyValue (
            DestroyFunc destroyValue )
```

Set the Destroy Value.

Parameters

destroyValue	The new destroy value function.

Returns

DestroyFunc The old destroy value function.

The documentation for this class was generated from the following files:

- ADTVector.hpp
- ADTVector.cpp

4.10 vectorNode Class Reference

Class for the vector Node object.

```
#include <ADTVector.hpp>
```

Public Member Functions

- vectorNode ()

  Construct a new vector Node object.
- **vectorNode** (Pointer value)
- Pointer getValue () const

  Get the Value object.
- void setValue (Pointer value)

  Set the Value object.

4.10.1 Detailed Description

Class for the vector Node object.

A vector node is described only its value.

4.10.2 Constructor & Destructor Documentation

4.10.2.1 vectorNode()

```
vectorNode::vectorNode ( )  [inline]
```

Construct a new vector Node object.

Parameters

value	A Pointer to the value.

4.10.3 Member Function Documentation

4.10.3.1 getValue()

```
Pointer vectorNode::getValue ( ) const  [inline]
```

Get the Value object.

Returns

Pointer

4.10.3.2 setValue()

```
void vectorNode::setValue (
            Pointer value )  [inline]
```

Set the Value object.

Parameters

value	

The documentation for this class was generated from the following file:

- ADTVector.hpp

Chapter 5

File Documentation

5.1 ADTGraph.hpp File Reference

Abstract undirected graph with weighted edges.

```
#include "ADTList.hpp"
#include "ADTMap.hpp"
#include "ADTVector.hpp"
```

Include dependency graph for ADTGraph.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Graph
- class GraphVertexPair

5.1.1 Detailed Description

Abstract undirected graph with weighted edges.

Author

Phaedon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

5.2 ADTGraph.hpp

Go to the documentation of this file.

```cpp

#pragma once

#include "ADTList.hpp"
#include "ADTMap.hpp"
#include "ADTVector.hpp"

class Graph {
 private:
  Vector* vec;
  Map* map;
  int size;
  CompareFunc compare;
  DestroyFunc destroy;
  HashFunc hash;

 public:
  Graph(CompareFunc compare, DestroyFunc destroy);
  ~Graph();
  int getSize();
  void insertVertex(Pointer vertex);
  List* getVertices();
  void removeVertex(Pointer vertex);
  void insertEdge(Pointer vertex1, Pointer vertex2, int weight);
  void removeEdge(Pointer vertex1, Pointer vertex2);
  int getWeight(Pointer vertex1, Pointer vertex2);
  List* getAdjacent(Pointer vertex);
  // Map* shortestPathLengths();
  void setHashFunction(HashFunc hash);
  CompareFunc getCompare() { return this->compare; };
  DestroyFunc getDestroy() { return this->destroy; };
  HashFunc getHash() { return this->hash; };
};

class GraphVertexPair {
 public:
  GraphVertexPair(Graph* owner, Pointer vertex1, Pointer vertex2)
      : vertex1(vertex1), vertex2(vertex2), owner(owner){};
  Pointer getVertex1() { return this->vertex1; };
  Pointer getVertex2() { return this->vertex2; };
  Graph* getOwner() { return this->owner; };

 private:
  Pointer vertex1;
  Pointer vertex2;
  Graph* owner;
};
```

5.3 ADTList.hpp

```cpp

#pragma once
#include "common.hpp"

#define LIST_BOF (ListNode*)0
#define LIST_EOF (ListNode*)0

class ListNode {
 public:
  ListNode() : next(nullptr), value(nullptr){};
  ListNode(Pointer value) : next(nullptr), value(value){};
  void setNext(ListNode* next);
  ListNode* getNext();
  Pointer getValue();

 private:
  ListNode* next;
  Pointer value;
};

class List {
 public:
  List(DestroyFunc value = nullptr);
  ~List();
  void insertNext(ListNode* node, Pointer value);
  void removeNext(ListNode* node);
  Pointer find(Pointer value, CompareFunc compare);
  DestroyFunc setDestroyValue(DestroyFunc value);
```

```
    int getSize();
    int increaseSize();
    int decreaseSize();
    ListNode* getHead();
    ListNode* getTail();
    ListNode* next(ListNode* node);
    Pointer nodeValue(ListNode* node);
    ListNode* findNode(Pointer value, CompareFunc compare);

private:
    DestroyFunc destroyValue;
    int size;
    ListNode* head;
    ListNode* tail;
};
```

5.4 ADTMap.hpp File Reference

`#include "common.hpp"`
Include dependency graph for ADTMap.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class MapNode
- class Map

Macros

- #define **MAX_LOAD_FACTOR** 0.5
- #define **MAP_EOF** (MapNode*)0

Enumerations

- enum **State** { **EMPTY** , **OCCUPIED** , **DELETED** }

5.4.1 Detailed Description

Author

Phaedon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

5.5 ADTMap.hpp

Go to the documentation of this file.

```

#pragma once

#include "common.hpp"

#define MAX_LOAD_FACTOR 0.5
#define MAP_EOF (MapNode*)0

typedef enum { EMPTY, OCCUPIED, DELETED } State;

class MapNode {
 public:
  MapNode() : key(nullptr), value(nullptr), state(EMPTY){};
  MapNode(Pointer key, Pointer value, State state)
      : key(key), value(value), state(state){};
  ~MapNode(){};
  void setKey(Pointer key) { this->key = key; };
  void setValue(Pointer value) { this->value = value; };
  void setState(State state) { this->state = state; };
  Pointer getKey() { return this->key; };
  Pointer getValue() { return this->value; };
  State getState() { return this->state; };

 private:
  Pointer key;
  Pointer value;
  State state;
};

class Map {
 public:
  Map();
  Map(CompareFunc compare, DestroyFunc destroyKey, DestroyFunc destroyValue);
  ~Map();
  void insert(Pointer key, Pointer value);
  bool remove(Pointer key);
  Pointer find(Pointer key);
  void rehash();
  void setCapacity(int capacity) { this->capacity = capacity; };
  void setDestroyKey(DestroyFunc destroyKey) { this->destroyKey = destroyKey; };
  void setDestroyValue(DestroyFunc destroyValue) {
    this->destroyValue = destroyValue;
  };
  void setHashFunction(HashFunc hash) { this->hash = hash; };
  MapNode** getArray() { return this->array; };
  int getCapacity() { return this->capacity; };
  int getSize() { return this->size; };
  int getDeleted() { return this->deleted; };
  DestroyFunc getDestroyKey() { return this->destroyKey; };
  DestroyFunc getDestroyValue() { return this->destroyValue; };
  HashFunc getHashFunction() { return this->hash; };
  MapNode* getFirst();
  MapNode* getNext(MapNode* node);
  MapNode* findNode(Pointer key);

 private:
  MapNode** array;
  CompareFunc compare;
  DestroyFunc destroyKey;
  DestroyFunc destroyValue;
  HashFunc hash;
  int capacity;
  int size;
  int deleted;
};
```

5.6 ADTPQueue.hpp File Reference

`#include "ADTVector.hpp"`

Include dependency graph for ADTPQueue.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class PQueue

  ADT Priority Queue.

5.6.1 Detailed Description

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

Copyright (c) 2023

5.7 ADTPQueue.hpp

Go to the documentation of this file.

```
#pragma once

#include "ADTVector.hpp"

class PQueue {
 public:
  PQueue(CompareFunc compare, DestroyFunc destroyValue, Vector* values);
  ~PQueue();
  int getSize();
  Pointer getMax();
  void insert(Pointer value);
  void removeMax();
  DestroyFunc setDestroyValue(DestroyFunc destroyValue);

  // Helper functions
  // These are used because the node IDs are 1-based, where as the
  // vector is 0-based.
  Pointer nodeValue(int nodeId);
  void nodeSwap(int nodeId1, int nodeId2);
  void bubbleUp(int nodeId);
  void bubbleDown(int nodeId);
  void naiveHeapify(Vector* values);

 private:
  Vector* vector;
  CompareFunc compare;
  DestroyFunc destroyValue;
};
```

5.8 ADTVector.hpp File Reference

ADTVector implementation using a dynamic array.

```
#include "common.hpp"
```

Include dependency graph for ADTVector.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class vectorNode

    Class for the vector Node object.
- class Vector

    ADT Vector.

Macros

- #define **VECTOR_MIN_CAPACITY** 10
- #define **VECTOR_BOF** (vectorNode*)0
- #define **VECTOR_EOF** (vectorNode*)0

5.8.1 Detailed Description

ADTVector implementation using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

5.9 ADTVector.hpp

Go to the documentation of this file.

```
#pragma once

#include "common.hpp"

#define VECTOR_MIN_CAPACITY 10

#define VECTOR_BOF (vectorNode*)0
#define VECTOR_EOF (vectorNode*)0

class vectorNode {
 public:
  vectorNode(){};
  vectorNode(Pointer value) : value(value){};
  Pointer getValue() const { return value; };
  void setValue(Pointer value) { this->value = value; };

 private:
  Pointer value;
};

```

```
class Vector {
 public:
  Vector(int size, DestroyFunc destroyValue);
  ~Vector();
  int getSize();
  void insertLast(Pointer value);
  int removeLast();
  Pointer getAt(int pos);
  int setAt(int pos, Pointer value);
  Pointer find(Pointer value, CompareFunc compare);
  DestroyFunc setDestroyValue(DestroyFunc destroyValue);
  vectorNode* first();
  vectorNode* last();
  vectorNode* next(vectorNode* node);
  vectorNode* previous(vectorNode* node);
  Pointer nodeValue(vectorNode* node);
  vectorNode* findNode(Pointer value, CompareFunc compare);

 private:
  vectorNode* array;
  int size;
  int capacity;
  DestroyFunc destroyValue;
};
```

5.10 common.hpp

```
#pragma once

typedef void* Pointer;

typedef int (*CompareFunc)(Pointer a, Pointer b);

typedef void (*DestroyFunc)(Pointer value);

typedef unsigned int (*HashFunc)(Pointer);
```

5.11 cppdescent.hpp

```

class cppdescent {
 public:
  int hello(void);
};
```

5.12 ADTGraph.cpp File Reference

```
#include "cppdescent/ADTGraph.hpp"
#include <climits>
#include <iostream>
```

Include dependency graph for ADTGraph.cpp:

5.13 ADTList.cpp File Reference

Implementation of an ADT linked list.

```
#include "cppdescent/ADTList.hpp"
```

Include dependency graph for ADTList.cpp:

5.13.1 Detailed Description

Implementation of an ADT linked list.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

5.14 ADTMap.cpp File Reference

Implementation of ADTMap.

```
#include "cppdescent/ADTMap.hpp"
#include <iostream>
```

Include dependency graph for ADTMap.cpp:

Variables

- int prime_sizes [ ]

5.14.1 Detailed Description

Implementation of ADTMap.

Author

Phaedon Seitanidis

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

5.14.2 Variable Documentation

5.14.2.1 prime_sizes

```
int prime_sizes[]
```

Initial value:

```
= {
    53,        97,        193,       389,       769,       1543,     3079,
    6151,      12289,     24593,     49157,     98317,     196613,   393241,
    786433,    1572869,   3145739,   6291469,   12582917,  25165843, 50331653,
    100663319, 201326611, 402653189, 805306457, 1610612741}
```

We want the capacity of the Hash Table to be a prime number, according to the theory. So the following list consists of prime numbers that are proven good capacity values for a Hash Table. When a rehash is needed, the capacity of the new table will be chosen from this table. If more than 1610612741 elements are needed, we double the capacity of the old table at rehash.

5.15 ADTPQueue.cpp File Reference

An ADT Priority Queue implemented using a heap.

```
#include "cppdescent/ADTPQueue.hpp"
#include <iostream>
```

Include dependency graph for ADTPQueue.cpp:

5.15.1 Detailed Description

An ADT Priority Queue implemented using a heap.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

Copyright (c) 2023

5.16 ADTVector.cpp File Reference

Implementation of ADTVector using a dynamic array.

```
#include "cppdescent/ADTVector.hpp"
#include "stdlib.h"
```

Include dependency graph for ADTVector.cpp:

5.16.1 Detailed Description

Implementation of ADTVector using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

Copyright (c) 2023

5.17 cppdescent.cpp File Reference

Implementation of cppdescent library.

```
#include "cppdescent/cppdescent.hpp"
#include <iostream>
```

Include dependency graph for cppdescent.cpp:

5.17.1 Detailed Description

Implementation of cppdescent library.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-27

Copyright

Copyright (c) 2023

5.18 listNode.cpp File Reference

Implementation of the listNode to be used with linkedList.

```
#include "cppdescent/ADTList.hpp"
```

Include dependency graph for listNode.cpp:

5.18.1 Detailed Description

Implementation of the listNode to be used with linkedList.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright